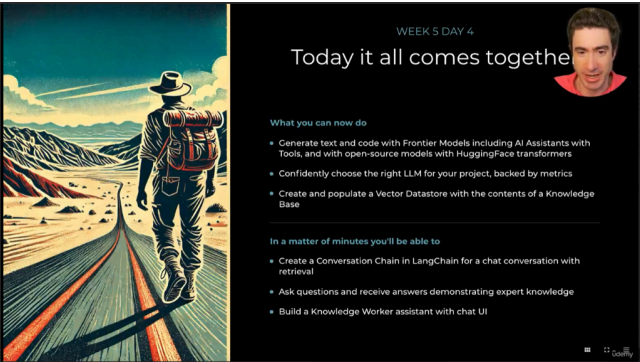




Xây dựng RAG pipeline thực tế:

- Sau khi đã học về các khái niệm như vectors, prompts và context, bạn sẽ được thực hành xây dựng một RAG (Retrieval-Augmented Generation) pipeline hoàn chỉnh.
- RAG là một kỹ thuật quan trọng cho phép mô hình ngôn ngữ lớn (LLM) truy xuất thông tin từ nguồn dữ liệu bên ngoài để cải thiện chất lượng câu trả lời.

Sử dụng LangChain để tạo Conversation Chain:

- Bạn sẽ sử dụng LangChain để tạo một Conversation Chain, giúp kết nối các thành phần khác nhau của RAG pipeline lại với nhau.
- Conversation Chain cho phép LLM duy trì ngữ cảnh trong cuộc trò chuyện và cung cấp câu trả lời phù hợp dựa trên lịch sử hội thoại.

Tạo hệ thống hỏi đáp với kiến thức chuyên môn:

- Bạn sẽ học cách xây dựng một hệ thống hỏi đáp có khả năng trả lời các câu hỏi phức tạp dựa trên kiến thức chuyên môn.
- Hệ thống này sẽ sử dụng RAG để truy xuất thông tin liên quan từ cơ sở dữ liệu vector và sử dụng thông tin đó để tạo ra câu trả lời chính xác.

Xây dựng Knowledge Worker Assistant với giao diện chat:

- Bạn sẽ xây dựng một trợ lý ảo Knowledge Worker Assistant với giao diện chat thân thiện.
- Trợ lý này có thể trả lời các câu hỏi của người dùng dựa trên kiến thức chuyên môn được cung cấp.

Sự dễ dàng khi sử dụng LangChain:

- Nhờ vào những kiến thức đã học trước đó, bạn sẽ thấy việc xây dựng RAG pipeline và các ứng dụng liên quan trở nên dễ dàng hơn nhiều.



Các khái niệm trừu tượng chính trong LangChain:

- LangChain định nghĩa một số khái niệm trừu tượng để đơn giản hóa việc xây dựng các ứng dụng LLM.
- Ba khái niệm chính được sử dụng trong bài học này là:
 - **LLM (Large Language Model):** Đại diện cho một mô hình ngôn ngữ lớn, ví dụ như OpenAI.
 - **Retriever:** Giao diện để truy xuất thông tin từ một nguồn dữ liệu, ví dụ như vector store (Chroma).
 - **Memory:** Đại diện cho lịch sử hội thoại với chatbot, giúp duy trì ngữ cảnh.

LLM:

- Trong trường hợp này, LLM đại diện cho OpenAI, nhưng có thể là các mô hình khác.
- LangChain cung cấp một đối tượng duy nhất để tương tác với mô hình.

Retriever:

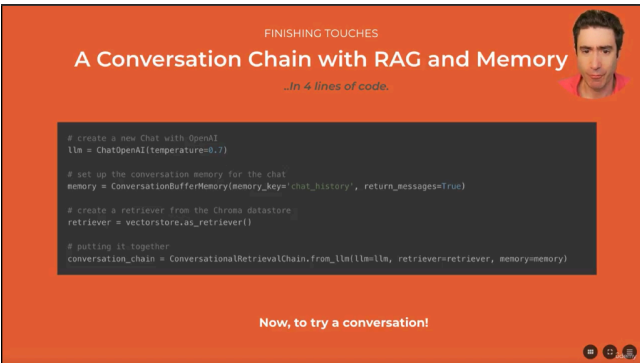
- Retriever là giao diện để truy xuất thông tin từ vector store (Chroma).
- Nó được sử dụng trong RAG (Retrieval-Augmented Generation) để lấy thông tin liên quan từ vector store và làm phong phú thêm prompt.

Memory:

- Memory đại diện cho lịch sử hội thoại, giúp chatbot duy trì ngữ cảnh.
- Nó trừu tượng hóa việc quản lý lịch sử hội thoại, ví dụ như danh sách các tin nhắn hệ thống, người dùng và trợ lý.
- LangChain tự động xử lý định dạng lịch sử hội thoại phù hợp với từng mô hình.

Xây dựng RAG pipeline:

- Với ba khái niệm trừu tượng này, việc xây dựng RAG pipeline sẽ trở nên đơn giản hơn.

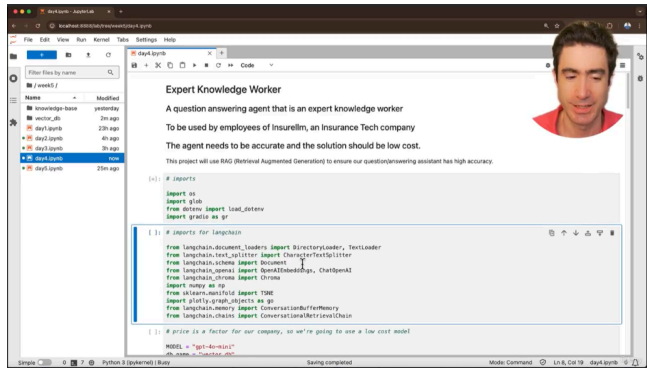


Xây dựng RAG pipeline với 4 dòng code: Bạn sẽ học được cách xây dựng một pipeline RAG (Retrieval Augmented Generation) chỉ với 4 dòng code bằng cách sử dụng LangChain.

Các thành phần của RAG pipeline:

- **LLM:** Khởi tạo một đối tượng LLM (ví dụ: ChatOpenAI) để tương tác với mô hình ngôn ngữ.
- **Memory:** Thiết lập bộ nhớ cho cuộc trò chuyện bằng cách sử dụng ConversationBufferMemory để lưu giữ lịch sử trò chuyện.
- **Retriever:** Tạo một retriever từ vector store (ví dụ: Chroma) để truy xuất các vector liên quan.
- **Conversation Chain:** Kết hợp LLM, retriever và memory để tạo thành một chuỗi hội thoại có khả năng truy xuất thông tin từ nguồn bên ngoài.

Thực hành trong JupyterLab: Bạn sẽ được hướng dẫn thực hành xây dựng và thử nghiệm RAG pipeline trong môi trường JupyterLab.



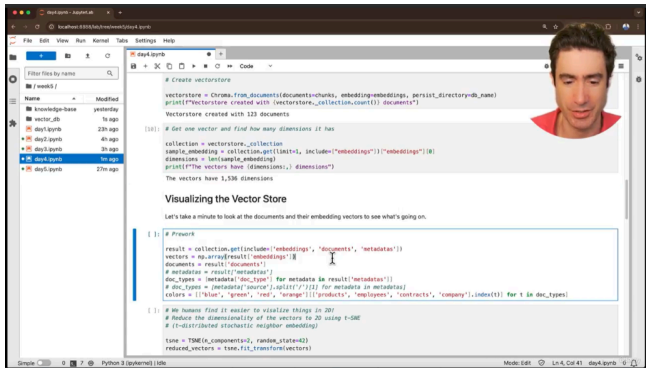
Xây dựng ứng dụng RAG (Retrieval-Augmented Generation) thực tế: Bạn sẽ học cách xây dựng một ứng dụng RAG hoàn chỉnh để tạo ra một trợ lý hỏi đáp thông minh.

Sử dụng LangChain: Bạn sẽ sử dụng thư viện LangChain để kết nối các thành phần khác nhau của ứng dụng RAG.

Các thành phần của ứng dụng RAG:

- **Tải dữ liệu:** Tải dữ liệu từ cơ sở tri thức (knowledge base) để cung cấp thông tin cho trợ lý.
- **Tạo chunks văn bản:** Chia nhỏ dữ liệu thành các chunks văn bản nhỏ hơn để xử lý hiệu quả hơn.
- **Sử dụng ChatOpenAI:** Sử dụng mô hình ChatOpenAI để tạo ra câu trả lời dựa trên thông tin được truy xuất.
- **Sử dụng ConversationBufferMemory:** Lưu trữ lịch sử trò chuyện để duy trì ngữ cảnh.
- **Sử dụng ConversationalRetrievalChain:** Kết hợp tất cả các thành phần trên để tạo ra một chuỗi hội thoại có khả năng truy xuất thông tin từ cơ sở tri thức.

Mục tiêu: Xây dựng một trợ lý hỏi đáp có kiến thức chuyên môn về công ty bảo hiểm công nghệ Insurelim.



Xây dựng RAG pipeline: học cách xây dựng một pipeline RAG (Retrieval Augmented Generation) để trả lời các câu hỏi dựa trên thông tin từ cơ sở dữ liệu vector.

Sử dụng LangChain: Bạn sẽ sử dụng LangChain để kết nối các thành phần khác nhau của pipeline RAG.

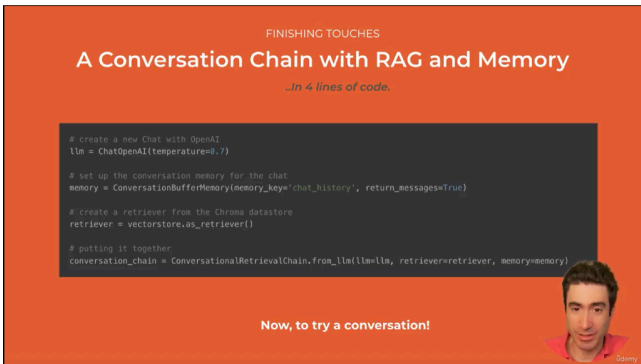
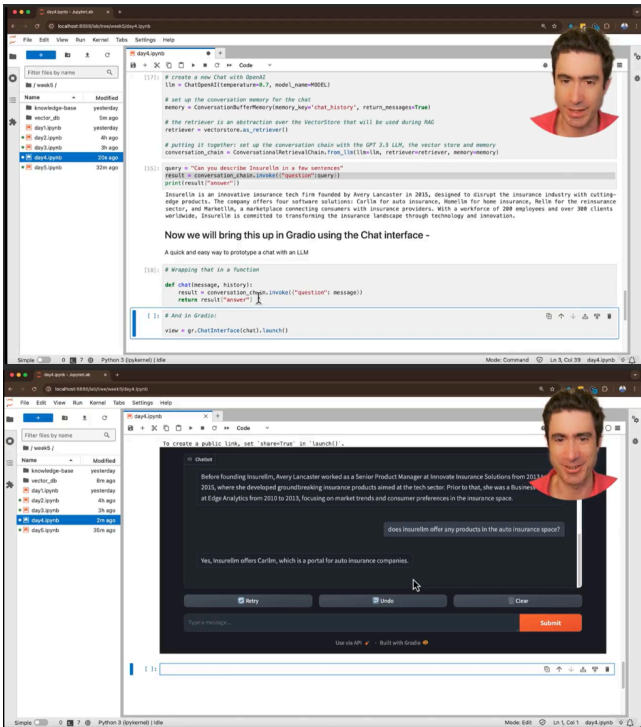
Các thành phần của RAG pipeline:

- **Tạo đối tượng LLM (ChatOpenAI):** Khởi tạo mô hình ChatOpenAI để tạo ra câu trả lời.
- **Tạo bộ nhớ (ConversationalBufferMemory):** Lưu trữ lịch sử trò chuyện để duy trì ngữ cảnh.
- **Tạo retriever:** Truy xuất các vector liên quan từ cơ sở dữ liệu vector (Chroma).
- **Tạo chuỗi hội thoại (ConversationalRetrievalChain):** Kết hợp các thành phần trên để tạo ra một chuỗi hội thoại có khả năng truy xuất thông tin từ cơ sở dữ liệu vector.

Thực hiện truy vấn: Bạn sẽ học cách thực hiện truy vấn bằng cách sử dụng phương thức invoke của chuỗi hội thoại.

Kết quả: Kết quả trả về sẽ chứa câu trả lời được tạo ra bởi mô hình dựa trên thông tin truy xuất từ cơ sở dữ liệu vector.

Tạo giao diện người dùng (Gradio): Bạn sẽ được giới thiệu về cách tạo giao diện người dùng đơn giản bằng Gradio để tương tác với pipeline RAG.



Tạo giao diện trò chuyện với Gradio:

- Bạn sẽ học cách sử dụng Gradio để tạo giao diện trò chuyện cho ứng dụng RAG của mình.
- Gradio cho phép bạn dễ dàng tạo giao diện người dùng để tương tác với các mô hình máy học.

Xử lý lịch sử trò chuyện:

- LangChain tự động xử lý lịch sử trò chuyện, vì vậy bạn không cần phải tự quản lý nó trong mã Gradio.
- LangChain duy trì ngữ cảnh của cuộc trò chuyện thông qua bộ nhớ (memory).

Thử nghiệm với ứng dụng RAG:

- Bạn được khuyến khích thử nghiệm với ứng dụng RAG bằng cách đặt các câu hỏi khó và xem liệu nó có thể trả lời chính xác hay không.
- Bạn có thể thử các câu hỏi về thông tin chi tiết về nhân viên, sản phẩm hoặc các khía cạnh khác của công ty.

Kiểm tra khả năng truy xuất thông tin:

- Bạn có thể kiểm tra xem ứng dụng RAG có thể truy xuất thông tin chính xác từ cơ sở dữ liệu vector hay không, ngay cả khi câu hỏi được diễn đạt theo những cách khác nhau.
- Ví dụ, hỏi về các sản phẩm về xe hơi của công ty, mà không cần dùng từ "car" hoặc tên sản phẩm.

Gỡ lỗi và cải thiện:

- Bạn sẽ được giới thiệu về các phương pháp gỡ lỗi và cải thiện hiệu suất của ứng dụng RAG.
- Bạn sẽ học cách tìm ra các vấn đề phổ biến với các lời nhắc và cách khắc phục chúng.

Tóm tắt về cách xây dựng RAG pipeline:

- Nhấn mạnh lại sự đơn giản của việc xây dựng RAG pipeline với LangChain.
- Chỉ cần 3 khái niệm trừu tượng (LM, Memory, Retriever) và 1 phương thức (ConversationRetrievalChain.from_llm) là đủ.

Các thành phần của RAG pipeline:

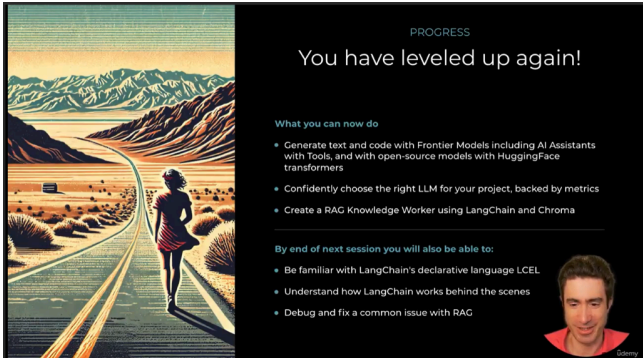
- LM (Large Language Model):** Mô hình ngôn ngữ lớn để tạo ra câu trả lời.
- Memory:** Lưu trữ lịch sử hội thoại để duy trì ngữ cảnh.
- Retriever:** Truy xuất thông tin từ vector store (ví dụ: Chroma) để cung cấp ngữ cảnh cho LLM.

Cách sử dụng Conversation Chain:

- Gọi phương thức invoke của conversation chain với một dictionary chứa câu hỏi.
- Lấy câu trả lời từ khóa answer trong kết quả trả về.

Nâng cao kỹ năng:

- Nắm vững kiến thức và kỹ năng cần thiết để xây dựng ứng dụng RAG.



Nâng cao kỹ năng xây dựng RAG pipeline:

- Bạn đã đạt được trình độ chuyên nghiệp trong việc xây dựng RAG (Retrieval-Augmented Generation) pipeline.
- Bạn có thể tạo ra các trợ lý trí thức RAG cho các công ty thực tế, không chỉ là công ty hư cấu.

Sử dụng LangChain và Chroma:

- Bạn có thể sử dụng LangChain và Chroma để xây dựng các ứng dụng RAG.
- Bạn cũng có thể dễ dàng chuyển đổi sang các vector data store khác và sử dụng các mô hình khác ngoài OpenAI.

Các nội dung sẽ học trong buổi tiếp theo:

- Giới thiệu về ngôn ngữ khai báo của LangChain (LCEL).
- Tìm hiểu về cách LangChain hoạt động bên dưới.
- Chẩn đoán và khắc phục một vấn đề phổ biến với RAG.

Mục tiêu:

- Giúp bạn sử dụng RAG một cách hiệu quả trong các dự án sản xuất.
- Cung cấp cho bạn kiến thức sâu sắc về hoạt động bên trong của RAG